

UNIVERSIDAD AUTONOMA DE AGUASCALIENTES

**INGENIERIA EN COMPUTACION
INTELIGENTE**

**ADMINISTRACION DE SOFTWARE Y
PROYECTOS**



PLANIFICACION SPRINT 4

Arrioja Pizaña Cesar Emmanuel

Cruz Mora Ángel

Semestre VIII

Panel de Asignación y Comunicación Bidireccional

El propósito de esta iteración fue conectar el trabajo individual de las dos interfaces construidas previamente (Oficina y Repartidores) para habilitar una dinámica colaborativa y ágil. El desarrollo se concentró en codificar el **Panel de Asignación**, permitiendo que un miembro de la oficina asocie digitalmente una orden específica a un conductor determinado con unos pocos clics. Asimismo, se priorizó la reducción de la brecha de comunicación; se implementó la lógica de sincronización para asegurar que cualquier cambio en las especificaciones o direcciones realizado desde el escritorio de administración se propague de inmediato al dispositivo móvil del chofer en ruta, evitando llamadas telefónicas reiteradas o retrasos logísticos.

El objetivo de este módulo es permitir la comunicación bidireccional y reactiva entre el backend y las plataformas (Oficina/Conductores) mediante una arquitectura dirigida por eventos (*Event-Driven Architecture*).

1. **Productor (Endpoint 1):** Un endpoint HTTP GET de prueba que valida la conectividad con el cluster de Kafka inyectando un evento base en el topic `tasks.lifecycle.progress`.
2. **Consumidor y Distribuidor SSE (Endpoint 2):** Un endpoint que abre un canal de transmisión persistente (text/event-stream). Escucha de forma centralizada los eventos que el consumidor de Kafka extrae del broker y, mediante reglas de negocio (Roles), decide a qué clientes despacharlos en tiempo real.

Desglose de Componentes Clave

1. Endpoint de Validación / Ping de Kafka

- **Ruta Sugerida:** `app/api/kafka/ping/route.ts`
- **Propósito:** Actúa como *Smoke Test* de la infraestructura. Valida de inmediato si las credenciales, los brokers y el cliente productor de Kafka están operando correctamente sin depender de la UI o de un trigger complejo de órdenes.
- **Componente de Red:** force-dynamic asegura que la función no sea estática/cacheada durante el build de Next.js, ejecutándose siempre en el servidor en cada petición.

2. Endpoint de Conexión de Eventos del Servidor (SSE)

- **Ruta Sugerida:** `app/api/events/route.ts`
- **Propósito:** Centralizar las conexiones activas de la interfaz de usuario de la **Oficina** y de la aplicación móvil de los **Drivers (Conductores)**.

- **Lógica de Filtrado por Roles (Crítico para el negocio):**
 - OFFICE: Acceso global. Cualquier actualización operativa en el ciclo de vida del inventario o rutas se le envía en tiempo real para mantener el tablero de control actualizado.
 - DRIVER: Acceso restringido. El sistema verifica el `payload.driverId` o `payload.driver_id`. Si coincide con el `userId` de la sesión SSE, el evento se envía; de lo contrario, se descarta silenciosamente para proteger la privacidad de datos y optimizar el ancho de banda del dispositivo móvil.

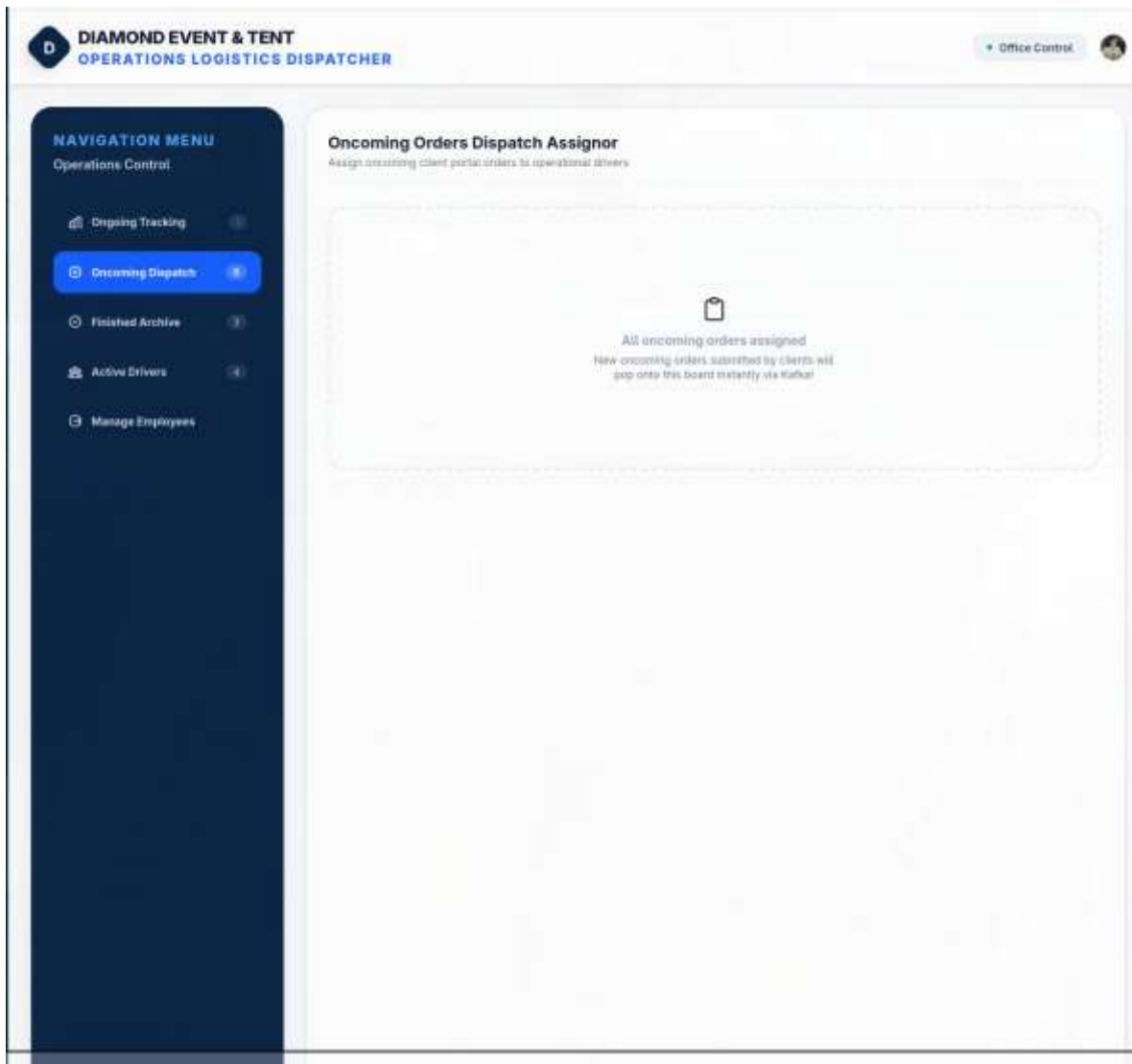
Consideraciones Técnicas y de Estabilización para este sprint

- Para asegurar que este backend no sufra caídas o fugas de memoria (*Memory Leaks*) bajo entornos de producción o desarrollo local, debes asegurar los siguientes puntos en los archivos importados:
- **Patrón Singleton en el Consumidor de Kafka (`startKafkaConsumer`):** Dado que Next.js levanta y destruye contextos en las API Routes (hot-reloading en desarrollo), `startKafkaConsumer()` debe implementar internamente una verificación para asegurarse de que **solo exista una instancia del consumidor activa** compartiendo el mismo `groupId`, evitando reconexiones infinitas o bloqueos de partición en Kafka.
- **El Bus de Eventos (`serverEvents`):** `serverEvents` debe ser una instancia global de un `EventEmitter` de Node.js. Al desconectarse un cliente (`req.signal.addEventListener("abort")`), el código ejecuta correctamente:

`“serverEvents.off("kafka-event", onKafkaEvent);”`

ENTREGABLES

Evidencia:



Código:

```
import { NextResponse } from "next/server";
import { produceEvent } from "@src/lib/kafka/client";

export const dynamic = "force-dynamic";

export async function GET() {
  try {
    const timestamp = new Date().toISOString();
    console.log(`[Kafka Test Ping] Producing ping event at ${timestamp}...`);

    await produceEvent(
      "tasks.lifecycle.progress",
      "test.ping",
```

```

    {
      message: "Kafka broker connectivity verified!",
      timestamp
    }
  );

  console.log("[Kafka Test Ping] Event published successfully.");

  return NextResponse.json({
    success: true,
    message: "Test ping event successfully produced to Kafka topic tasks.lifecycle.progress!",
    timestamp
  });

} catch (error: any) {
  console.error("[Kafka Test Ping Error] Failed to publish event:", error);
  return NextResponse.json({
    success: false,
    error: error.message || String(error)
  }, { status: 500 });
}
}

`Este codigo es otra funcion para los eventos del Sprint 4`

import { NextRequest } from "next/server";
import { serverEvents } from "@src/lib/kafka/events";
import { startKafkaConsumer } from "@src/lib/kafka/consumer";

export const dynamic = "force-dynamic";

export async function GET(req: NextRequest) {
  const { searchParams } = new URL(req.url);
  const userId = searchParams.get("userId");
  const role = searchParams.get("role"); // 'OFFICE' | 'DRIVER'

  if (!userId || !role) {
    return new Response(JSON.stringify({ error: "Missing userId or role" }), {
      status: 400,
      headers: { "Content-Type": "application/json" }
    });
  }

  console.log(`[SSE Connection] Client connected: userId=${userId}, role=${role}`);

  // Ensure Kafka consumer is up and listening
  // It runs in the background as a singleton
  startKafkaConsumer().catch(err => {
    console.error("Error starting Kafka Consumer in SSE connection:", err);
  });

  const encoder = new TextEncoder();

  const stream = new ReadableStream({
    async start(controller) {
      // 1. Send initial handshake connection event
      const initMessage = `data: ${JSON.stringify({ type: "connection", message: "SSE Connection Established"
    })}\n\n`;
      controller.enqueue(encoder.encode(initMessage));
    }
  });

```

```

// Keepalive heartbeat interval to prevent gateway timeouts (e.g. AWS ALB, NGINX, Cloudflare)
const heartbeat = setInterval(() => {
  try {
    controller.enqueue(encoder.encode(": heartbeat\n\n"));
  } catch (err) {
    // Client disconnected
    clearInterval(heartbeat);
  }
}, 15000);

// 2. Define the event listener for incoming Kafka events
const onKafkaEvent = (data: { topic: string; key: string; event: any }) => {
  try {
    const { topic, event } = data;
    const { payload } = event;

    let shouldPush = false;

    if (role === "OFFICE") {
      // Office sees all operational event updates
      shouldPush = true;
    } else if (role === "DRIVER") {
      // Drivers only receive events related to their assignments
      if (payload && (payload.driverId === userId || payload.driver_id === userId)) {
        shouldPush = true;
      }
    }

    if (shouldPush) {
      // console.log(`[SSE Push] Pushing event to ${userId} (${role}) on topic ${topic}`);
      const sseData = `data: ${JSON.stringify({ topic, event })}\n\n`;
      controller.enqueue(encoder.encode(sseData));
    }
  } catch (err) {
    console.error(`[SSE Push Error] Failed to push to client ${userId}:`, err);
  }
};

// Register the event emitter listener
serverEvents.on("kafka-event", onKafkaEvent);

// 3. Handle connection close and cleanup
req.signal.addEventListener("abort", () => {
  console.log(`[SSE Connection] Client disconnected: userId=${userId}`);
  clearInterval(heartbeat);
  serverEvents.off("kafka-event", onKafkaEvent);
  try {
    controller.close();
  } catch (e) {}
});
});

return new Response(stream, {
  headers: {
    "Content-Type": "text/event-stream",
    "Cache-Control": "no-cache, no-transform",
    "Connection": "keep-alive",
  },
});

```